INTERNAL · AUTHORIZED SECURITY ASSESSMENT PLATFORM

VAPT-AI V2

Project Status & Technical Report — architecture, multi-agent orchestration, tooling, governance, progress to date, and the delivery plan to 1 December 2026.

Report date	2026-10-17
Linear project	VAPT-AI V2 (Internal Beta) · `P-BAR-1`
Codebase	`/vapt-ai-v2` — monorepo (FastAPI · Next.js · LangGraph · Celery)
Beta window	2026-10-08 → 2026-11-19 (plan extended to 2026-12-01 in this report)
Overall status	4 / 6 milestones complete ~76% of issues done
Scope	Web · Network · White-box source. **Mobile excluded** (decision 2026-10-08).

Authorized use only. The platform enforces deny-by-default scope and policy in code (`packages/policy` , `packages/tools`). Every intrusive action is human-gated. This report documents a defensive, governed assessment system intended to run only against explicitly authorized targets.

Contents

1. 1. Executive summary & scorecard
2. 2. Progress measured against Linear
3. 3. System architecture
4. 4. Workflow & multi-agent graph (diagram)
5. 5. The AI orchestrator & every agent
6. 6. Governed execution & human-in-the-loop
7. 7. CVE pipeline & Tenable / Nessus intel
8. 8. Tooling inventory (built-ins + 65-tool catalog)
9. 9. Phases executed before a report is produced
10. 10. Skills: current state & how to add them
11. 11. Multi-model: moving beyond a single Qwen
12. 12. Open-source tools to review & integrate
13. 13. Delivery plan 2026-10-17 → 2026-12-01
14. 14. Risks & recommendations

1 · Executive summary

VAPT-AI V2 is a **governed, multi-agent security-assessment engine** rebuilt from the V1 prototype into a clean monorepo. A human defines an engagement (scope + allowed action classes); a LangGraph orchestrator plans and runs a bounded set of tools against in-scope assets, matches findings to CVEs, corroborates them with authenticated-scanner data (Tenable.sc / Nessus), and produces an evidence-linked report. **A model only proposes; a deterministic policy engine decides; a human approves anything intrusive.**

76%	4/6	10	94/126
LINEAR ISSUES DONE (26/34)	MILESTONES AT 100%	ORCHESTRATED AGENT PHASES	PENTEST TOOLS INSTALLED

WHAT IS BUILT & WORKING

- Auth (Argon2id + revocable sessions, RBAC), FastAPI API, Next.js UI
- MariaDB + transactional outbox + Redis/Celery durable run queue
- LangGraph engagement graph with 10 phases, pause/resume checkpoints
- Deterministic P0–P3 policy engine, argv-only tool broker, execution ledger
- Recon/scan/validate + CVE pipeline + Tenable/Nessus intel
- White-box source graph (semgrep / gitleaks / dep-audit)
- Active web-vuln & approved-PoC flows (human-gated), HTML/PDF reports
- Inline approval dock, live streaming ledger, 44 tests

WHAT REMAINS (FOCUS OF THIS PLAN)

- **Week 5** — cancellation & recovery, report-review workflow & model routing, platform security review
- **Week 6** — defect fixes & lab acceptance, performance/reliability, backup/restore, deployment package & runbooks
- **Enhancements requested:** real multi-model routing (today it is one Qwen for all roles), a reusable *skills* layer, and reviewed open-source tool integrations

> **Bottom line:** the hard engineering (governance core, orchestration, tool mesh, evidence→report path) is done and tested. The remaining ~6.5 weeks are about *operational robustness, result quality, and depth* — exactly where multi-model routing, skills and curated OSS integrations convert "it runs" into "it produces a solid, defensible result."

2 · Progress measured against Linear

Source: Linear project **VAPT-AI V2 (Internal Beta)** (`P-BAR-1`), 34 issues across 6 weekly milestones. Weeks 1–4 and Week 3 (Web & network) are fully closed; Weeks 5–6 are not started.

Milestone completion

Milestone	Target	Progress	Exit gate
Week 1 — Foundations	2026-10-15	100%	Scoped run persists a task, runs one safe tool, completes one model call ✅
Week 2 — Controlled execution	2026-10-22	100%	Allowed actions execute; approval pauses & resumes; rejection blocks ✅
Week 3 — Web & network	2026-10-29	100%	Black/gray-box lab cases end-to-end with normalized evidence links ✅
Week 4 — Source (white-box)	2026-11-05	100%	Source static runs, limited approved PoC, draft evidence-linked reports ✅
Week 5 — Operations & migration	2026-11-12	0%	Feature freeze; migration rehearsal + security-review results ⏳
Week 6 — Hardening & release	2026-11-19	0%	Signed acceptance + internal beta deployment; operator/recovery guides ⏳

Note: Weeks 3 & 4 were closed ahead of their target dates (both completed 2026-10-13/14), so the team is **running ahead of the original schedule** on delivered scope.

Issue status

DONE (26) — HIGHLIGHTS

ID	Title
BAR-5..14	Monorepo, API & contracts, auth, MariaDB, Redis/Celery, LLM gateway, LangGraph skeleton, V1 triage
BAR-15..20	Agent harness, policy engine, tool broker, ledger, approval inbox, pause/resume
BAR-21..26	Recon, Nmap/Nuclei/ZAP, Nessus/Tenable, Burp MCP, Validation+CVE, evidence viewer
BAR-27..30	Source static analysis, report/remediation, schedules & retest, approved-PoC flow
BAR-39..44	Real multi-agent LangGraph + gateway, outbox relay, approval dock, non-default ports, gated SQLi/web-vuln

BACKLOG (8) — ALL IN WEEKS 5–6

ID	Title	Wk
BAR-31	Cancellation & recovery	5
BAR-32	Legacy V1 data import & redaction	5
BAR-33	Report review workflow & model routing	5
BAR-34	Security review of the platform	5
BAR-35	Defect fixes & lab acceptance	6
BAR-36	Performance & reliability checks	6
BAR-37	Backup / restore & recovery evidence	6
BAR-38	Deployment package & operator runbooks	6

> **Reading the number:** "76% of issues" understates maturity because the remaining 8 are operational/hardening, not core capability. But it *also* hides that **BAR-33 (model routing)** is the hook for the multi-model work you asked about — it is already on the board for Week 5.

3 · System architecture

A modular monolith with separate processes for **API**, **orchestration** and **tools/workers**, communicating through a durable queue and a transactional outbox so runs are idempotent and resumable.

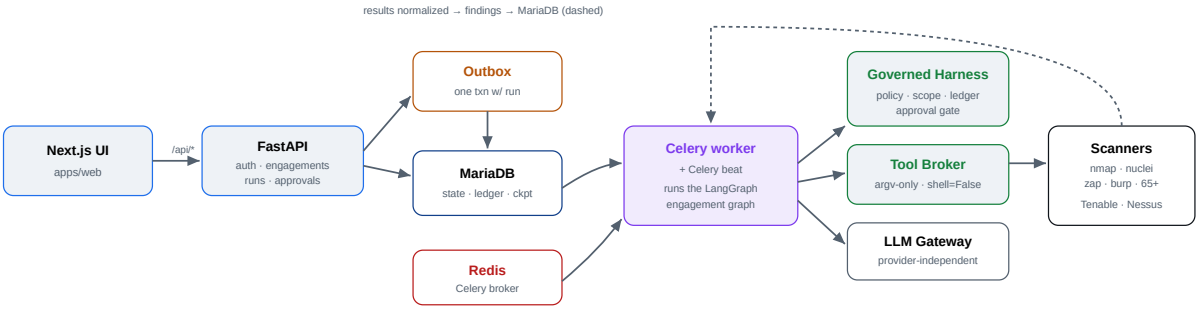


Fig 1 — Process topology. The API writes a run and an outbox row in one transaction; beat relays queued runs to the worker; the worker executes the LangGraph graph; every tool call passes the Governed Harness → Tool Broker.

PACKAGES (PURE LOGIC, NO FRAMEWORK COUPLING)

Package	Role
`domain`	Enums + Pydantic contracts
`persistence`	SQLAlchemy models, ledger, web creds
`policy`	Deterministic P0–P3 decisions
`tools`	Registry + 65-tool catalog + argv broker
`harness`	GovernedHarness (one mandatory path)
`llm_gateway`	Provider-independent `invoke()` + profiles
`agents`	Manifests, planner, phases, toolrouter, CVE, report, playbooks

SECURITY INVARIANTS ENFORCED IN CODE

1. **Deny-by-default**: out-of-scope & cloud-metadata targets denied with no approval path; P3 denied; P2 always needs approval.
2. **No shell**: tools run as validated argv arrays; option-like targets rejected.
3. **No self-signed tokens**: Argon2id + opaque revocable sessions (hash-at-rest).
4. **Durable, idempotent runs**: outbox + stable idempotency keys; re-delivery resumes the same thread.
5. **Candidate ≠ confirmed**: findings carry status; validation is a separate step.
6. **Secrets by reference**: LLM/Tenable keys via env, never in the ledger.

4 · Workflow & multi-agent graph

The engagement is a **LangGraph state machine**. Each node is an agent/phase. Self-looping nodes (scan, webvuln, attack) process **one action per visit** so an `interrupt()` for approval never drifts the resume value. Committed findings live in the DB — the source of truth that survives any pause.

agent / phase node · human-in-the-loop (approval gate) · passive / auto (P0–P1) · intrusive (P2, gated) · ↻ self-loop (one action/visit)

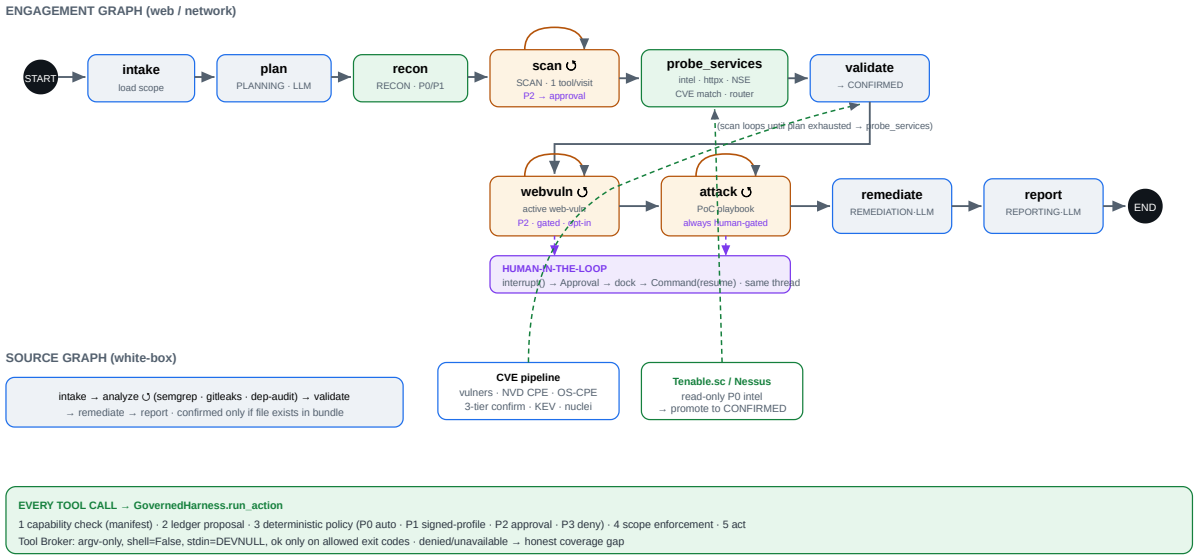


Fig 2 — The multi-agent engagement graph with agents, model-backed nodes (LLM), self-loops (↻), human approval gates, the Tenable/Nessus intel feed, the CVE pipeline, the white-box source graph, and the single governed execution path every tool must pass through.

5 · The AI orchestrator & every agent

Each node runs under a **versioned manifest** (`packages/agents/manifests.py`): a declared contract that can only *narrow* what the platform allows. The manifest is a second, agent-scoped gate in front of the deterministic policy engine — it decides what an agent may even *ask for*; it never authorizes.

WHAT A MANIFEST BOUNDS

- **allowed_tool_ids** — exact tools the agent may propose (anything else is refused even if the registry would allow the role).
- **risk_ceiling** — highest `PolicyClass` it may propose (a recon agent can never propose P2).
- **state_reads / state_writes** — which `EngagementState` fields it may touch (can't mutate scope or another agent's output).
- **budgets** — `max_iterations` , wall-seconds, tokens, cost; a breach raises `ManifestError` → recorded escalation, never silent continuation.
- **model_profile** — which gateway profile it may call; **redaction_keys**; **escalation_conditions**.

Agent (manifest)	Phase node	Model profile	Risk ceiling	Budget	Responsibility
PLANNING `planning@1`	plan	planning (T=0)	P1	1 iter · 4k tok · $0.05	Proposes an ordered, bounded scan plan as schema-constrained JSON. Model proposes; every step re-validated against registry + scan manifest + ceiling. Falls back to a deterministic default plan offline.
RECON `recon@1`	recon	— (no LLM)	P1	4 iter · 600s	Establishes reachability (passive, bounded). DNS skipped for a bare IP.
SCAN `scan@1`	scan, probe_services, webvuln	— (deterministic)	P2	16 iter · 3600s	Runs planned tools + service-driven router through the harness; emits candidates. May *propose* intrusive; policy + human still gate. PoC tools are deliberately excluded here.
VALIDATE `validate@1`	validate	validation (T=0)	P1	32 iter · 1800s · 2k tok	Independently re-checks each candidate; the **only** step that marks CONFIRMED. CVE candidates go through the 3-tier confirmation.
ATTACK `attack@1`	attack	— (no LLM)	P2 always escalate	4 iter · 900s	Opt-in only. Runs one narrow, named PoC playbook per confirmed finding; **every PoC is forced through human approval** regardless of policy class.
REMEDIATION `remediation@1`	remediate	reporting (T=0.2)	P0	1 iter · 4k tok · $0.10	Drafts remediation grounded strictly on confirmed findings. Runs no tools. Deterministic note offline.
REPORTING `reporting@1`	report	reporting (T=0.2)	P0	1 iter · 8k tok · $0.20	Assembles the evidence-linked HTML/PDF report draft. Grounded (titles/severities only, no secrets). Degrades offline.

> **Separation of powers (the core principle).** A model only ever proposes a `tool_id` + typed args. It can never widen a decision. The **policy engine** decides; the **harness** enforces scope; a **human** approves anything intrusive. Three grounding guards keep LLM output honest: schema-constrained planning, re-validation of every proposed step, and a strict "never invent CVEs/ports/hosts" system prompt for remediation/reporting.

Model-call auditing

Every gateway call is recorded in a `model_calls` ledger table — profile, provider, model id, finish reason, token usage, latency, trace id, agent id, prompt version — **counts only, never prompt or output text**. This is what makes the multi-model routing in §11 measurable per agent.

6 · Governed execution & human-in-the-loop

Every tool call goes through **one** mandatory path: `GovernedHarness.run_action`. There is no second way to execute a tool.

THE 5-STEP PATH

1. **Capability check** — may this agent use this tool? (manifest)
2. **Proposal recorded** in the ledger (idempotent on an action digest)
3. **Deterministic policy decision** by class (below)
4. **Scope enforcement** — target must fall inside engagement domains/CIDRs; `local` tools act on the uploaded bundle
5. **Act** — ALLOW → broker executes · REQUIRE_APPROVAL → create approval + `interrupt()` · DENY → record a coverage gap

POLICY CLASSES

Class	Decision
P0 passive	Always allowed (read-only intel, passive checks)
P1 low-impact	Auto under a **signed profile** (lab); else approval
P2 intrusive	**Always** needs human approval
P3 excluded	Never runs (catalogued, policy-denied)

Absolute denials: out-of-scope targets and cloud-metadata addresses (169.254.169.254, metadata.*) — no approval path at all.

Where the human is in the loop (as built today)

Gate	Class	Trigger	Mechanism
Any P2 scan action not pre-allowed	P2	policy REQUIRE_APPROVAL	graph `interrupt()` → pending Approval → dock
Windows aggressive checks (ms08-067, ms12-020)	P2	crash-risky NSE, only if P2 enabled	human-gated in `probe_services`
Active web-vuln pass (SQLi etc.)	P2	`webvuln` node, opt-in	per-tool approval, one interrupt/visit
Every PoC / exploitation playbook	always	ATTACK manifest always-escalate	forced approval regardless of class

When policy returns REQUIRE_APPROVAL the **whole graph pauses**. A global floating **approval dock** in the UI polls pending approvals and lets a reviewer approve/reject inline (tool, target, reason, enriched detail). On approval the worker resumes the same thread via `Command(resume=…)`; a stale/duplicate resume is a safe no-op (guarded by the checkpoint's pending state). Durable checkpoints mean a paused run survives a worker restart.

> **Observability:** the graph is driven with per-super-step commits, so the Execution Ledger fills phase-by-phase live. Each tool publishes `action.executing` (with a "running Ns" counter) and `action.executed` ("took Ns"). Denied/unavailable/non-ok tools are recorded as **coverage gaps** so the report never claims coverage it did not achieve.

7 · CVE pipeline & authenticated-scanner intel

Findings become CVEs through several independent matchers, then a 3-tier confirmation. Only the **validate** phase marks a finding CONFIRMED.

MATCHING (PRODUCES CANDIDATES)

- **nmap_vuln (vulners)** — service→CVE from `-sV --script vulners`
- **NVD CPE fallback** — product+version → NVD CVEs (works when vulners is unreachable; version required)
- **OS-CPE leads** — detected Windows build → top-critical CVEs, *precision-gated* to where the OS CPE is the vulnerable component ("patch unverified")
- **Windows NSE** — ms17-010 (safe, P1, auto); ms08-067 + ms12-020 (crash-risky, P2, gated)

CONFIRMATION — 3 TIERS

1. **CONFIRMED** — a safe on-target NSE check reported VULNERABLE, *or* the CVE's nuclei template fired (nuclei skipped for OS-CPE leads)
2. **EXPLOIT_AVAILABLE** — not confirmed on-target but public intel exists: **CISA KEV** or an NVD exploit-tagged reference
3. **INCONCLUSIVE** — version/OS match only, no template hit, no known exploit

CVE details are enriched from **NVD** (CVSS, description, references), cached 24h. `VAPT_NVD_API_KEY` raises the rate limit 5→50 req/30s. Statuses: `candidate → confirmed | exploit_available | inconclusive`; severity bands follow NVD CVSS thresholds.

External authenticated-scanner intel (Tenable.sc + Nessus)

A **read-only (P0 passive)** evidence stream that corroborates the active scan. Runs first in `probe_services`, independent of the active scan or signed profile.

Source	Tool / shim	Behaviour
Tenable.sc (live)	`tenable_scan` → `workers/bin/vapt-tenable`	Queries existing Tenable.sc cumulative vuln data over REST per in-scope host. Creds via env or V1's `~/.vapt_ai/tenable.json`. **Does not scan** — serves what Tenable already has.
Nessus (offline)	`nessus_import` → `workers/bin/vapt-nessus-import`	Parses an exported `.nessus` file offline. No network.

Both emit a shared JSONL shape → `_intel` normalizer → `ingest_external_intel`: a CVE the active scan already found is **promoted to CONFIRMED** (not duplicated); a CVE only Tenable knows becomes a **new confirmed** finding; `exploit=true` is treated like KEV; info/low no-CVE items are skipped so the report isn't flooded.

> **Dependency to note for the plan:** authenticated-scanner depth currently rests on **commercial Tenable/Nessus**. §12 proposes adding **OpenVAS/Greenbone (GVM)** as an open-source authenticated scanner so the platform isn't licence-bound for that evidence tier.

8 · Tooling inventory

Two layers, both governed identically: ~20 **built-in registry tools** (bespoke parsers) and a declarative **65-tool catalog** (`packages/tools/catalog.py`) spanning all five phases. Governance (policy class, allowed agents, target kind) is encoded as *data* per row — adding a tool is adding a row. A tool whose binary is absent simply no-ops into an honest coverage gap.

~20	65	94/126	P0–P3
BUILT-IN REGISTRY TOOLS	CATALOG TOOLS (DATA ROWS)	INSTALLED ON THIS HOST	GOVERNANCE PER TOOL

BUILT-IN REGISTRY (BESPOKE NORMALIZERS)

`dns_resolve` , `http_probe` , `nmap_service_scan` , `nmap_vuln` , `searchsploit` , `nuclei_cve/safe/intrusive` , `win_vuln_scan` , `win_vuln_aggressive` , `host_scripts` , `wapiti` , `zap_baseline` , `tls_check` , `nessus_import` , `tenable_scan` , `semgrep_scan` , `gitleaks_scan` , `dep_audit` , `burp_request` , `poc_screenshot` .

CATALOG BY PHASE (65)

Phase	Count	Class
1 · Recon & asset discovery	12	P0
2 · Scanning & enumeration	14	P1
3 · Vulnerability assessment	16	P1
4 · Exploitation	15	P2 ATTACK-only
5 · Post-exploitation / relay	8	P2 / P3

REPRESENTATIVE TOOLS PER PHASE

Recon (P0)	subfinder, amass_passive, assetfinder, sublist3r, theharvester, whois, dnsrecon, waybackurls, gau, katana, whatweb, wafw00f
Scan/enum (P1)	naabu, masscan, gobuster_dir, ffuf, feroxbuster, dirsearch, arjun, smbmap, enum4linux_ng, smbclient_list, ldapsearch_base, cewl, onesixtyone, smtp_user_enum
Vuln (P1)	nikto, wpscan, joomscan, droopescan, cmseek, graphql_cop, shcheck, humble, sslyze, ssh_audit, subzy, swaks_relay, corsy, wapiti, xsstrike, xsrfprobe
Exploit (P2, gated)	sqlmap, ghauri, commix, sstimap, dotdotpwn, fuxploider, nosqlmap, hydra, medusa, responder, mitm6, bettercap, ettercap, beef, sliver
Post-exploit	netexec_smb, impacket_secretsdump, lsassy, bloodyad, certipy_find, chisel, ligolo_proxy, krbrelayx (relay/C2 → P3 denied)

The **service-driven tool router** (`toolrouter.py`) deterministically picks the right follow-up tools for what was actually discovered (web → whatweb/katana/nikto/feroxbuster; TLS → sslyze; SSH → ssh_audit; SMB → smbmap/enum4linux_ng; SMTP → swaks; LDAP → ldapsearch) so the catalog is exercised even when the LLM planner is offline. Run `python3 scripts/check_tools.py` to see installed vs missing (currently 94/126; missing include sliver, responder, bettercap, ettercap, pupy, routersploit).

9 · Phases executed before a report is produced

For a web/network engagement, the report at the end is the product of this ordered pipeline. Each row is a graph node; the ledger records a `phase.*` event as it completes.

#	Phase	Agent	Produces	Gated?
1	**intake**	—	Loads scope (domains/CIDRs, allowed classes, allowed ports)	no
2	**plan**	PLANNING (LLM)	Ordered bounded scan plan (model-proposed, re-validated; deterministic fallback)	no
3	**recon**	RECON	Reachable assets (passive)	no
4	**scan** ↻	SCAN	Candidate findings — one planned tool per visit (nmap `-sT -sV -Pn` etc.)	P2→yes
5	**probe_services**	SCAN	Tenable/Nessus intel ingest · httpx on odd web ports · NSE host scripts · Windows vuln checks · service→CVE · NVD/OS-CPE leads · tool router	P2→yes
6	**validate**	VALIDATE	CONFIRMED findings — independent re-check + 3-tier CVE confirmation (bounded, highest-severity first)	no
7	**webvuln** ↻	SCAN	Active web-vuln results (opt-in; wapiti/nuclei against seeded URLs)	yes
8	**attack** ↻	ATTACK	PoC evidence (opt-in; one playbook per applicable confirmed finding)	always
9	**remediate**	REMEDIATION (LLM)	Remediation guidance grounded on confirmed findings	no
10	**report**	REPORTING (LLM)	Evidence-linked HTML/PDF: exec summary, remediation, findings by severity, coverage gaps	no

White-box source engagements use a parallel graph: `intake → analyze ↻ (semgrep · gitleaks · dep-audit) → validate → remediate → report`, where a static finding is confirmed only if the file it points at exists in the uploaded bundle.

> **Report grounding:** remediation and reporting agents are constrained by a strict system prompt ("never invent vulnerabilities, CVEs, ports, services, versions, or hosts not explicitly listed") and degrade to a deterministic template when the model is offline — so a report is always factual and always produced, with or without an LLM.

10 · Skills — current state & how to add them

> **Finding:** the codebase uses **no "skills" abstraction today.** A grep for `skill` across `packages/`, `services/` and `apps/` returns nothing. What the system *does* have is three adjacent mechanisms that together cover part of what a skills layer would give you:

Existing mechanism	What it does	Gap vs a skills layer
Manifests	Per-agent capability contract (tools, risk ceiling, budgets, prompt version)	Bounds *what* an agent may do, not reusable *how-to* procedure/knowledge
Playbooks	Named bounded PoC → one registry tool (http_evidence, screenshot)	Only 2, exploitation-only; not a general per-task knowledge pack
Tool router	Deterministic service→tool selection	Hard-coded heuristics, no packaged methodology/prompt library

What "skills" should mean here

A **skill** = a versioned, reusable package of *procedural knowledge* that an agent loads on demand for a specific task: a focused instruction/prompt template, the exact tool sequence, parsing/grading rules, and worked examples. (This is the Anthropic "Agent Skills" model — progressive-disclosure folders of `SKILL.md` + resources — adapted to a local-model gateway.) Skills differ from manifests: a manifest is a *guardrail*; a skill is *know-how*.

How to include a skills layer (recommended design)

1. **New package** `packages/skills/` — each skill a folder: `skill.yaml` (id, version, trigger conditions, allowed tools, model profile, budget) + `instructions.md` (the methodology/prompt) + optional `grading.py` (how to judge output) + examples.
2. **Registry + loader** mirroring the tool catalog: skills are *data*, discovered at startup, each tied to an `AgentRole` and gated by the same manifest/policy rules (a skill can never widen tool access beyond the agent's manifest).
3. **Selection**: the planner/router picks a skill by trigger (e.g. "WordPress detected" → `wordpress-assessment` skill; "GraphQL endpoint" → `graphql-assessment`). The skill's `instructions.md` is injected into that node's system prompt (progressive disclosure — only the relevant skill loads).
4. **Grounding**: skill instructions are untrusted-data-safe (they can't redefine scope/policy), versioned like `prompt_version`, and auditable in the `model_calls` ledger.

FIRST SKILLS TO AUTHOR (HIGH ROI, MAP TO TOOLS YOU ALREADY HAVE)

- CMS assessment (WordPress/Joomla/Drupal) → wpscan/joomscan/droopescan/cmseek
- TLS/crypto review → sslyze/testssl + grading to CVSS bands
- Web content discovery → katana/feroxbuster/ffuf with sane wordlist policy
- SMB/AD enumeration → enum4linux_ng/smbmap/netexec (read-only)
- API/GraphQL assessment → graphql_cop/arjun/corsy
- Report-writing skill → per-severity narrative + remediation templates

Estimated effort for the skills framework + first 6 skills: see §13 (≈6 working days).

11 · Multi-model — moving beyond a single Qwen

> **Good news:** the scaffolding for multi-model is **already in the code**. Today all three profiles (planning, validation, reporting) resolve to one `VAPT_LLM_MODEL` (your Qwen), but the gateway was built to point each profile at a different model — you can enable it with environment variables, no code change for the basic case.

How the gateway is structured (today)

- **Task-shaped profiles**: `planning` (T=0), `validation` (T=0), `reporting` (T=0.2) — agents already call by profile, not by model.
- **Per-profile override already supported**: `config.py` reads `VAPT_LLM_<PROFILE>_MODEL` (e.g. `VAPT_LLM_PLANNING_MODEL`) and falls back to the shared model if unset.
- **Provider-independent**: `ollama` or any OpenAI-compatible server (vLLM, llama.cpp, LM Studio, TGI). Each profile builds its own client.
- **Fallback chains**: `ModelProfileConfig.fallback_profile` — on a real failure the gateway retries once on a fallback, *only* if that fallback is permitted for the same data class ("never widen data permissions").
- **Data-class guardrail**: a local-only engagement can never be silently routed to a hosted model.

Why multi-model helps (match model to task)

Profile / agent	What the task needs	Suggested model tier
planning	Strong structured-JSON + tool reasoning, low latency, deterministic	A capable instruct model tuned for tool/JSON (e.g. a larger Qwen-Coder / Llama-3.x-70B class, T=0)
validation	Careful, conservative judgement; cheap & fast (runs many times)	A small fast local model (e.g. Qwen-14B / 8B), T=0 — throughput matters
reporting / remediation	Fluent, well-structured prose grounded on findings	A prose-strong model (larger local, or a hosted model only if data class permits)

Enable it — two levels

LEVEL 1 — CONFIG ONLY (TODAY, ~0.5 DAY)

Point each profile at a different model/endpoint via env:

```
VAPT_LLM_PLANNING_MODEL=qwen2.5-coder:32b
VAPT_LLM_VALIDATION_MODEL=qwen2.5:14b
VAPT_LLM_REPORTING_MODEL=llama3.3:70b
```

Works now because `model_for(profile)` already reads these.

LEVEL 2 — PER-PROFILE ENDPOINT/PROVIDER (SMALL CODE CHANGE)

Extend `profiles_from_env()` so each profile can also override `provider`, `base_url`, `api_key_env` and `fallback_profile` (e.g. planning on a vLLM box, reporting on a hosted endpoint for standard-class data, validation on a tiny local model). Wire `fallback_profile` so planning/validation degrade to the reporting model rather than to the deterministic fallback.

> **This is BAR-33 ("Report review workflow & model routing").** It's already on the Week-5 board — the plan in §13 schedules the config layer first (immediate win) and the per-profile routing + an evaluation harness (compare models per task on a fixed lab target, using the existing `model_calls` ledger and `scripts/eval_planner.py`) right after.

Guardrails to keep when going multi-model

- Keep **data-class enforcement** strict — never let a fallback route local-only content to a hosted model.
- Keep planning/validation at **T=0**; only reporting gets latitude.
- Record every call in `model_calls` (already done) so model-vs-model quality is **measurable**, not anecdotal.
- Models still only **propose**; the policy engine and human gates are unchanged regardless of which model is behind a profile.

12 · Open-source tools to review & integrate

You already have a deep *scanner* mesh. The highest-value OSS additions are the ones that (a) remove a commercial dependency, (b) add a tier you don't have (prioritization, vuln management, report templating), or (c) deepen white-box. Each integrates through the existing governed catalog/adapters — a new tool is a new governed row, never a new execution path.

Tool (OSS)	Gap it fills	Integration shape	Effort	Priority
OpenVAS / Greenbone (GVM)	Open-source *authenticated* scanner — reduces Tenable/Nessus licence dependency for the corroboration tier (§7)	Read-only adapter like `tenable_scan`; emit the same JSONL intel shape → `ingest_external_intel`	3 d	High
OSV-Scanner + EPSS	Dependency CVEs from OSV + *exploit-prediction (EPSS)* scoring for prioritization	Catalog tool for OSV (white-box/dep-audit); EPSS as an enrichment call alongside NVD/KEV in the CVE pipeline	2 d	High
Trivy	Container image / IaC / SBOM / secret scanning for white-box bundles	Governed `local` catalog tool in the source graph's analyze phase (P1)	1.5 d	Med-High
DefectDojo (or Faraday)	Cross-engagement vulnerability management / dedup / trend — today findings live per-run only	Export adapter: push confirmed findings via API after `report`; optional import of history	3 d	Med
SysReptor	Professional pentest *report templating* (branded, repeatable deliverables)	Alternate reporting backend: feed grounded findings JSON → SysReptor template render	2.5 d	Med
Nuclei template mgmt	Keep templates fresh + add custom org templates (depth you already pay for with nuclei)	Scheduled `nuclei -update-templates` + a curated custom-template dir mounted into the broker	1 d	Med
CodeQL	Deep semantic SAST beyond semgrep for the white-box tier	Governed `local` analyze tool; heavier, so opt-in per source engagement	2.5 d	Opt
Kali MCP server (V1 asset)	You already have `MCP-Kali-Server` + Burp MCP in V1 — formalize as governed MCP tool adapters	MCP client adapter behind the broker; each exposed action is a governed catalog row	2 d	Med

> **Integration rule (keep it safe):** every new tool enters as a declarative catalog row with an explicit policy class, allowed agents, and argv template. Destructive/relay/C2 capability stays `P3` (denied) or `P2` ATTACK-only + human-gated. A new integration is reviewed the same way a playbook is — a deliberate, auditable expansion.

Total OSS-integration effort if all "High/Med-High/Med" items are taken: ≈ 15 working days, scheduled across Weeks B–E in §13 so they land without blocking the Week 5–6 release work.

13 · Delivery plan — now to 1 December 2026

The goal: reach a **solid, trustworthy V2** and launch the internal beta on **1 December 2026**. The work splits into three simple groups, running over the next seven weeks:

① LAUNCH WORK

The things that **must** be finished to ship safely: stopping/recovering runs, the report sign-off step, a security check of the platform, bug-fixing, backups, and the install package.

② SMARTER AI

Making the AI give better results: using **more than one model** (right model for each job) and adding reusable **skills** (how-to recipes for common situations).

③ NEW TOOLS

Adding carefully-chosen **free, open-source tools** that add new abilities (an open scanner, better vulnerability prioritisation, nicer reports).

The seven weeks at a glance

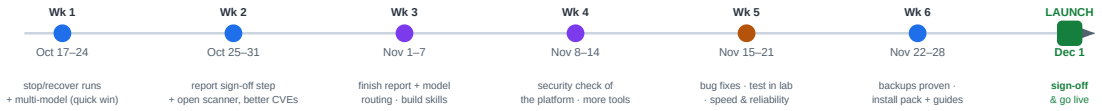


Fig 3 — The runway. Dot colour shows the main focus that week: blue = launch work, purple = smarter AI, orange = new tools. The green box is launch day.

Schedule (which weeks each piece of work is active)

Launch work (required to ship) · Smarter AI (multi-model & skills) · New tools (open-source) · Launch

Piece of work	Wk1 Oct17	Wk2 Oct25	Wk3 Nov1	Wk4 Nov8	Wk5 Nov15	Wk6 Nov22	Dec1
Stop & recover a run safely	■						
Report sign-off step (human approves report)		■	■				
Security check of the platform itself				■			
Bug fixes & testing in the lab					■	■	
Speed & reliability tuning					■		
Backups + proven restore						■	
Install package + operator guides						■	■
Use several models (quick on/off setting)	■						
Pick best model per job + compare them		■	■				
Skills system + first 6 skills			■	■			
Open scanner + better CVE ranking		■	■				
Container/code tools + finding tracker				■	■		
Nicer report templates					■		

A coloured cell means that work is happening that week. Plan assumes ~2 people. With only one person, do the blue "launch work" first and let the purple/orange items slide — none of them block the launch.

13 · Delivery plan — what each piece of work means

Launch work — required to ship

Piece of work	In plain words	Effort · when
Stop & recover a run (BAR-31)	A user can press **Stop** and a scan halts cleanly; and if the server crashes, a run picks up where it left off instead of being lost or starting over.	~3 days Wk 1
Report sign-off step (BAR-33)	A person reviews, edits and **approves** the AI-drafted report before it's final — the AI drafts, a human signs off.	~4 days Wk 2–3
Security check of the platform (BAR-34)	Audit the tool *itself* for weaknesses (logins, secrets handling, that the approval gates can't be bypassed) before it's trusted.	~4 days Wk 4
Bug fixes & lab testing (BAR-35)	Fix known problems, then run the whole platform against safe practice targets to **prove it finds the right things** and makes a correct report.	~5 days Wk 5–6
Speed & reliability (BAR-36)	Make sure it stays fast and stable on bigger jobs and doesn't fall over under load.	~3 days Wk 5
Backups + restore (BAR-37)	Automatic copies of all data, and a **tested, documented** way to bring everything back after a disaster.	~3 days Wk 6
Install package + guides (BAR-38)	A one-command install and plain operator manuals (how to run it, back it up, troubleshoot) so someone other than the developer can run it.	~3 days Wk 6

Smarter AI — the enhancements you asked for

Piece of work	In plain words	Effort · when
Use several models — quick setting	Today one model (Qwen) does every job. This flips on a setting that already exists so each job can use a **different, better-suited model** — no coding needed. Immediate quality win.	~½ day Wk 1
Pick the best model per job + compare	Let each job run on its own server/model with a backup, and build a small test rig to **measure which model is actually best** for each job — decisions on evidence, not guesswork.	~4 days Wk 2–3
Skills system + first 6 skills	Add reusable **how-to recipes** the AI loads for common situations (e.g. "WordPress site found → run these checks this way"). First six cover CMS, TLS, web discovery, Windows/SMB, APIs, and report-writing. Biggest lever on result quality.	~6 days Wk 3–4

New tools — open-source add-ons

Piece of work	In plain words	Effort · when
Open scanner + better CVE ranking	Add **OpenVAS** (a free scanner, so you're not tied to paid Tenable) and **OSV + EPSS** (rank vulnerabilities by how likely they are to be exploited).	~5 days Wk 2–3
Container/code tools + finding tracker	Add **Trivy** (scans containers & code), keep scan templates fresh, and push findings into **DefectDojo** so results are tracked across engagements, not just per run.	~5½ days Wk 4–5
Nicer report templates	Add **SysReptor** as an optional report backend for polished, branded, repeatable deliverables.	~2½ days Wk 5

"Effort" is rough working-days for the task itself; "when" is the week it's scheduled. Every new tool is added under the same safety rules as existing tools (locked to a risk level, dangerous capabilities stay switched off).

Checkpoints — what should be true by each date

By this date	What's done
Fri 24 Oct	Stop/recover a run works · the platform is using several models (quick setting on)
Fri 31 Oct	We can measure which model is best for each job · better CVE ranking live · open scanner being wired in
Fri 7 Nov	Report sign-off step done · skills system running with at least 3 skills · open-scanner results showing in reports
Fri 14 Nov	Platform security check finished · all 6 skills live · container/code tools + finding tracker connected
Wed 19 Nov	**Feature freeze** (original target) — no new features after this; bug-fixing + speed/reliability done
Fri 28 Nov	Backups + restore proven · install package and operator/recovery guides drafted
Mon 1 Dec	**Launch:** passes lab testing on all engagement types, multi-model + skills + new tools in use, evidence-linked reports, internal beta deployed and signed off ✅

14 · Risks & recommendations

RISKS TO MANAGE

Risk	Mitigation
Enhancement track crowds out release (Weeks 5–6)	Track R is the hard commit; Track Q items are independently shippable and can slip to post-beta without blocking release
Single-engineer capacity	Serialize Q behind R; drop CodeQL + SysReptor to post-beta (both marked Opt/Med)
Model quality drift across models	The `model_calls` ledger + a fixed-target eval make quality measurable before any model is promoted
Commercial-scanner dependency (Tenable)	OpenVAS/GVM adapter (Q4) gives an OSS authenticated-scan tier
OSS tool sprawl / unsafe capability	Every tool is a governed catalog row with an explicit policy class; relay/C2 stays P3-denied
Report hallucination	Already mitigated (grounded prompts + deterministic fallback); keep this invariant when swapping reporting models

RECOMMENDATIONS (PRIORITY ORDER)

1. **Ship the config-only multi-model win this week** — zero code, immediate per-task quality gain.
2. **Protect the Week 5–6 release path** — it is the difference between "a beta" and "a demo".
3. **Build the skills layer** — it is the biggest lever on *result quality*, and it reuses tools you already installed.
4. **Add OpenVAS + OSV/EPSS early** — authenticated-scan independence and exploit-aware prioritization materially improve findings.
5. **Keep the governance invariants frozen** — model/skill/tool changes must never touch the propose→decide→approve separation.
6. **Stand up the eval harness before promoting any model** — decisions on data, not vibes.

Definition of "solid V2" at 2026-12-01

- Black-, gray- and white-box lab cases complete end-to-end with normalized, evidence-linked findings and an exported PDF report.
- Multi-model routing live: planning / validation / reporting each on a task-matched model, measured by the eval harness.
- A working skills layer with ≥6 reviewed skills driving depth where tools already exist.
- OSS tier added: OpenVAS authenticated intel, OSV/EPSS prioritization, Trivy white-box, DefectDojo tracking.
- Operationally ready: cancellation/recovery, backup/restore proven, security review closed, deployment package + runbooks, signed acceptance.

Summary. The platform's governed core, multi-agent orchestration and evidence→report path are built and tested (76% of Linear closed, 4/6 milestones at 100%, running ahead of the original schedule). The next 6.5 weeks are about operational robustness and result quality — and the two capabilities you asked about (multi-model and skills) are both low-risk to add because the gateway already supports per-profile models and the governance model already gives a clean slot for a skills package. Hold the release track as the hard commitment; layer quality on top.

Report generated 2026-10-17 from Linear project P-BAR-1 and the `/vapt-ai-v2` codebase (docs/WORKFLOW.md, orchestrator graph, agents, policy, tools catalog, llm_gateway) and a live `scripts/check_tools.py` run (94/126 tools installed).